

cRIO UI in Minutes Using Only LabVIEW

OR, AN INTRO TO REMOTE PANEL SERVER

JOSEPH VON ARX (“JVA” ON LV DISCORD)

JULY 2026

Remote Panel Server

Client (PC) downloads and “runs” Front Panel(s).

Server (cRIO) runs the code (Block Diagram and SubVIs).

Remote Panel Server allows you to re-use your cRIO app's LabVIEW Front Panels as the main UI for the application.

No browser needed.

No 3rd party tooling needed.

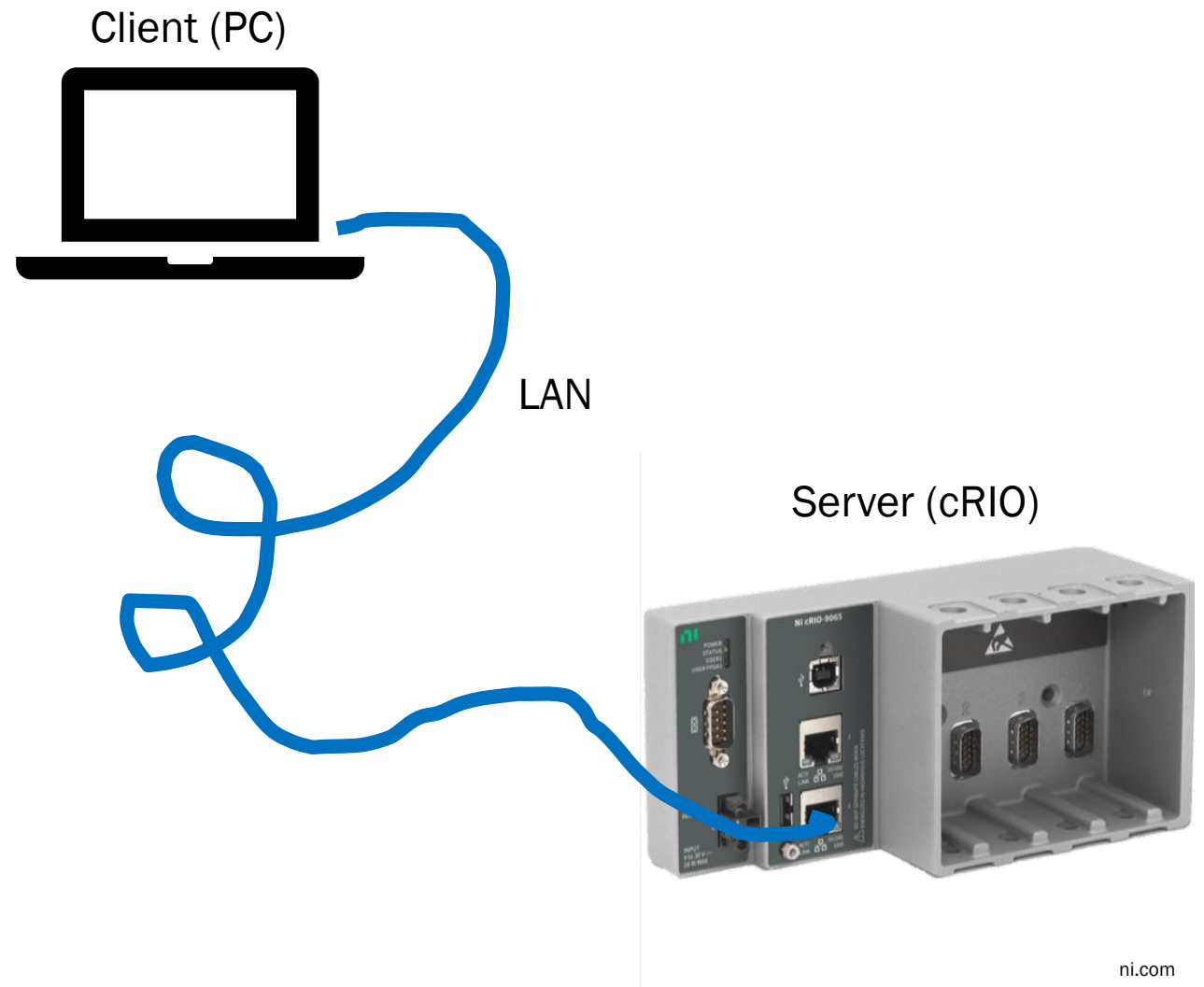
No need to maintain a secondary UI codebase.

Most of the cRIO developers I talk to do not know about this technique.

Knowing this technique would have saved me months of time back when I looking for options for cRIO UIs.

Hardware

- cRIO (Server)
- PC (Client)
- Both on the same LAN

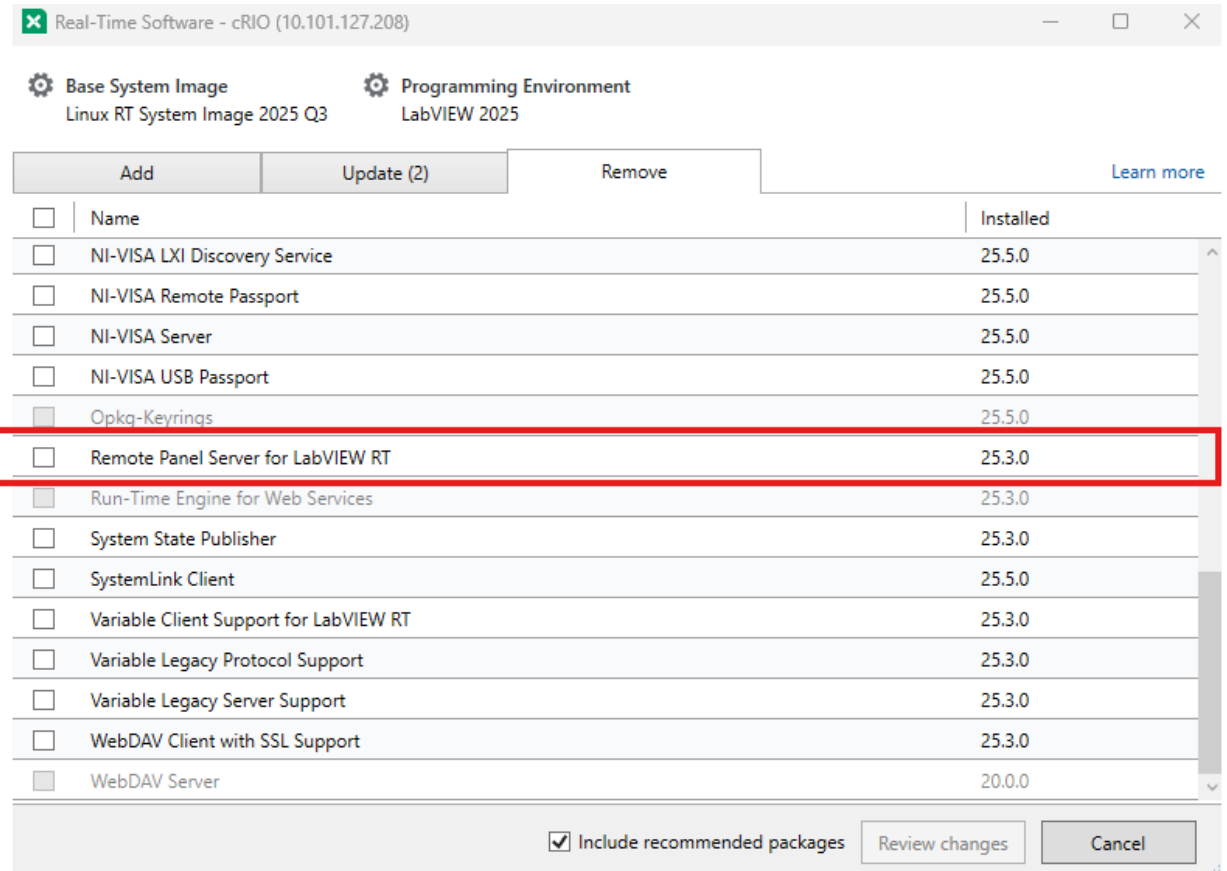


ni.com

Server (cRIO) Configuration

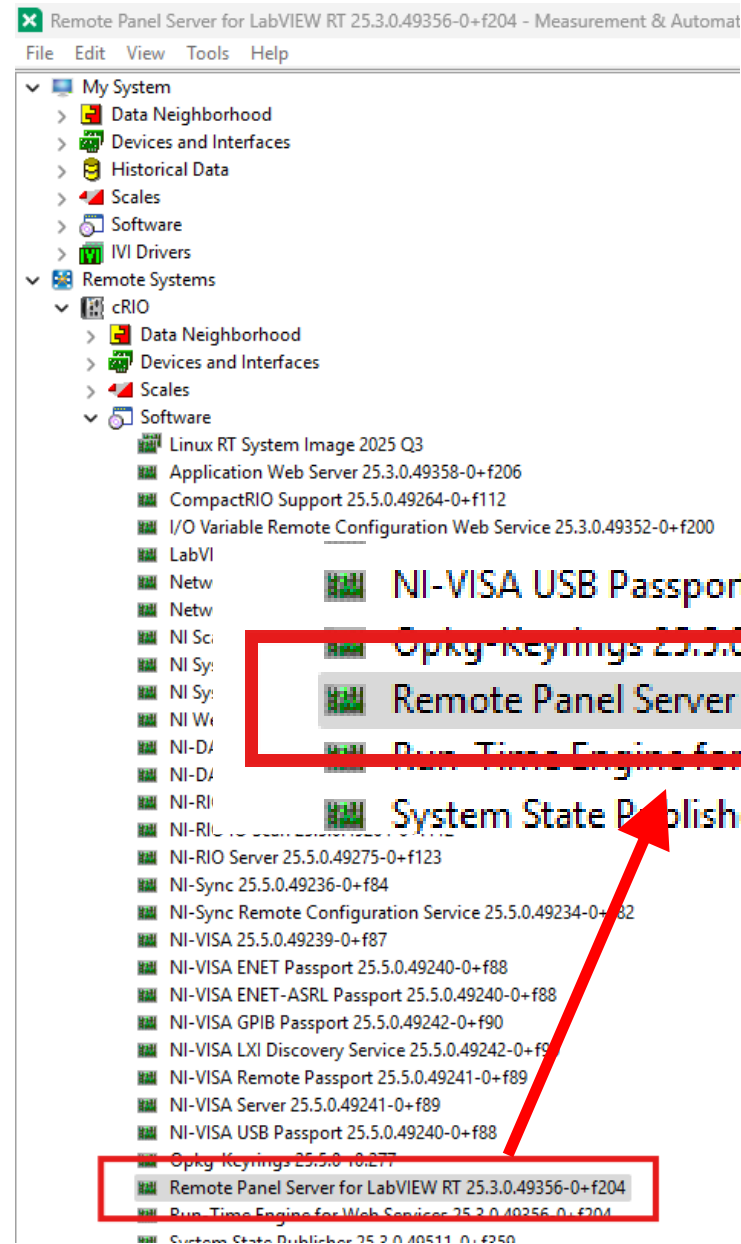
Enabling Remote Panels

Step #1. Install Remote Panel Server driver to cRIO



Enabling Remote Panels

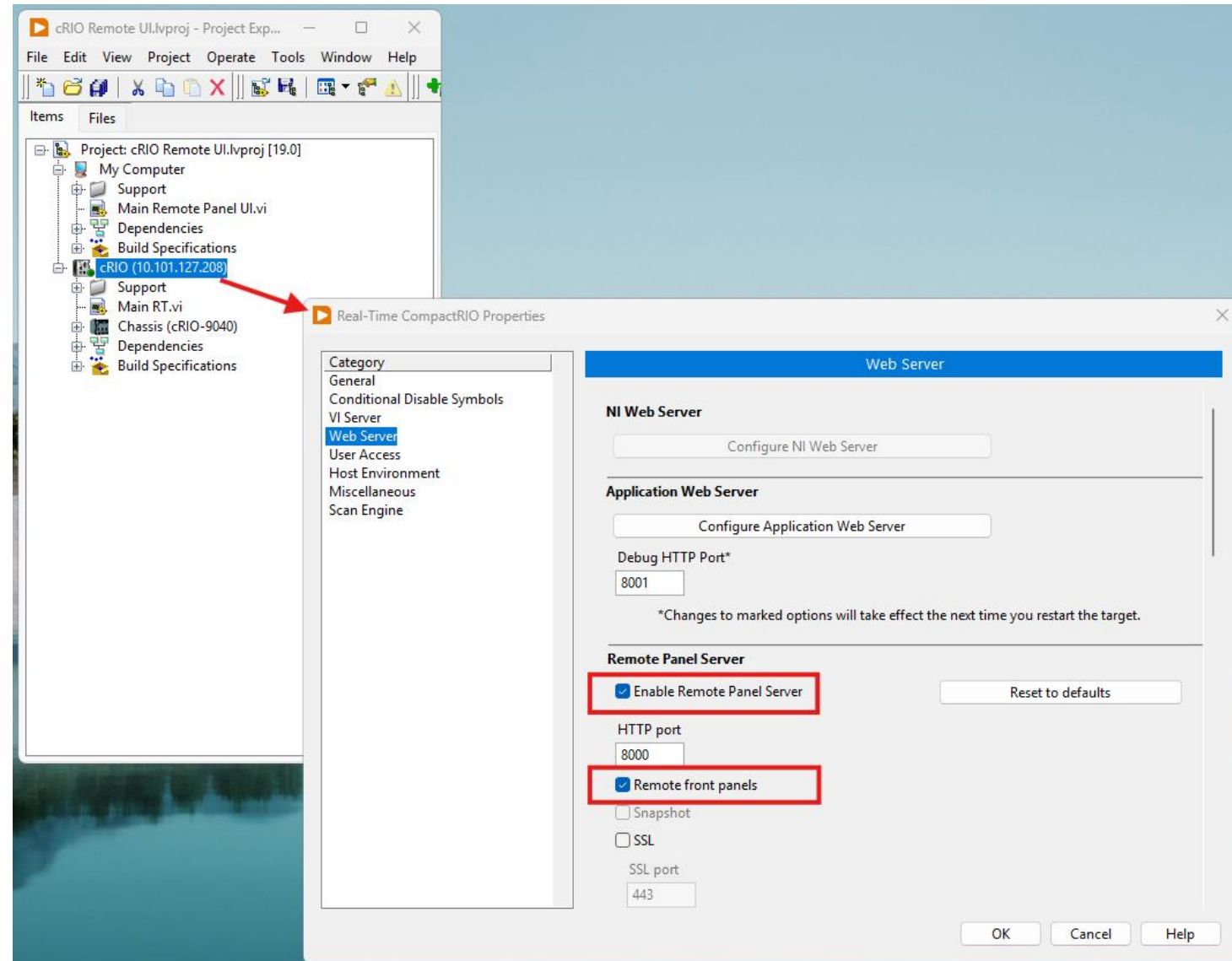
Step #1. Install Remote Panel Server driver to cRIO



(Included in the NI MAX baseline cRIO Software package but unselected by default)

Enabling Remote Panels

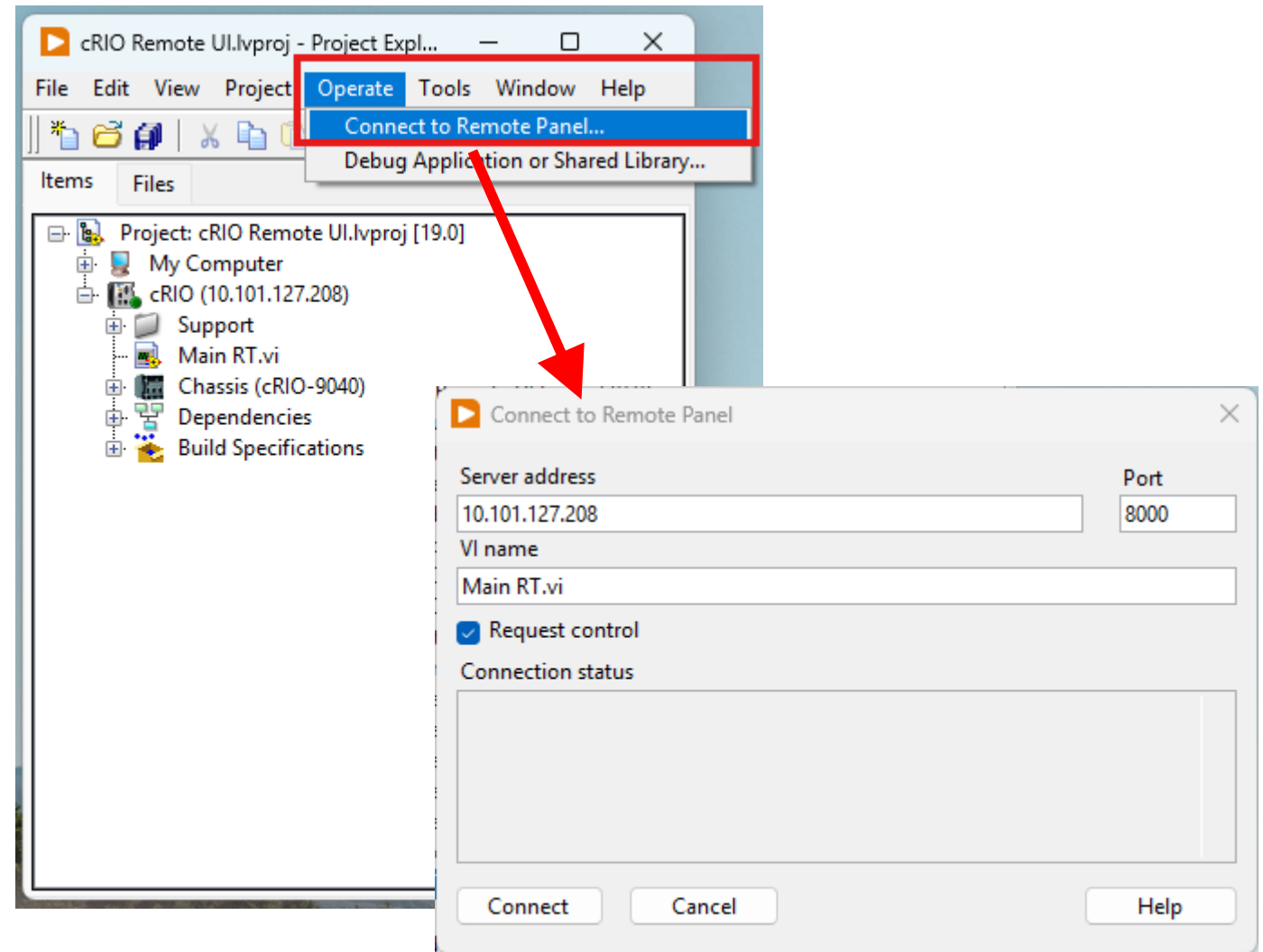
Step #2. Enable Remote Panel Server on cRIO target in your LabVIEW project



Client (PC) Options

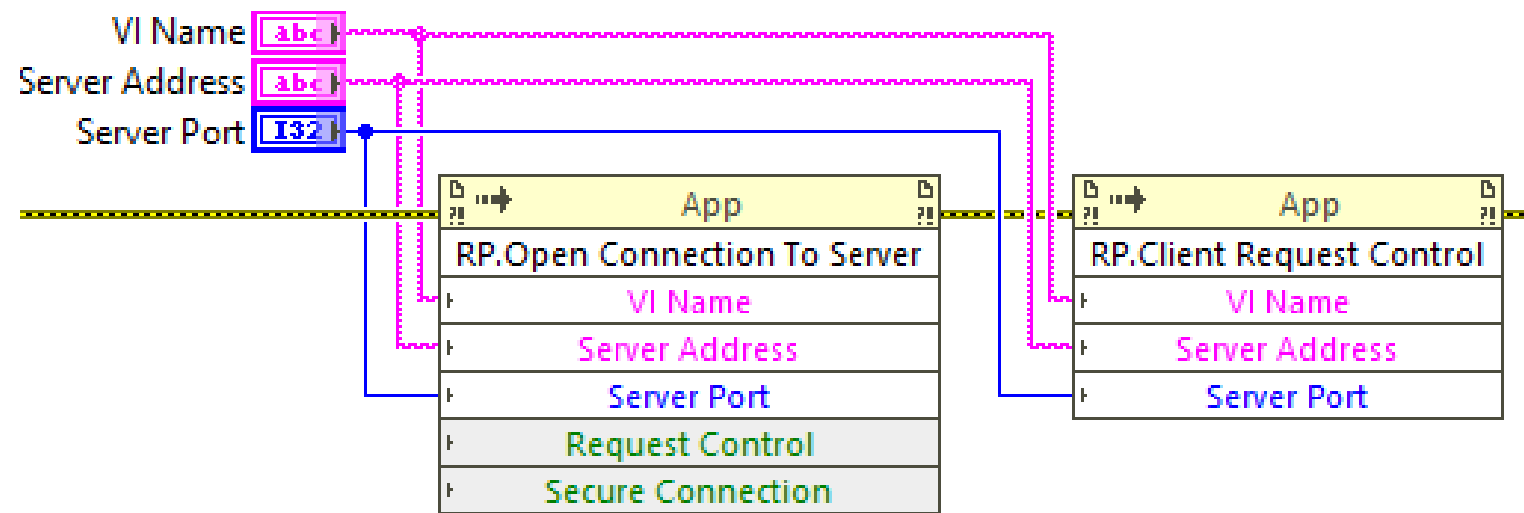
Connecting to Remote Panels

Option #1. From LabVIEW



Connecting to Remote Panels

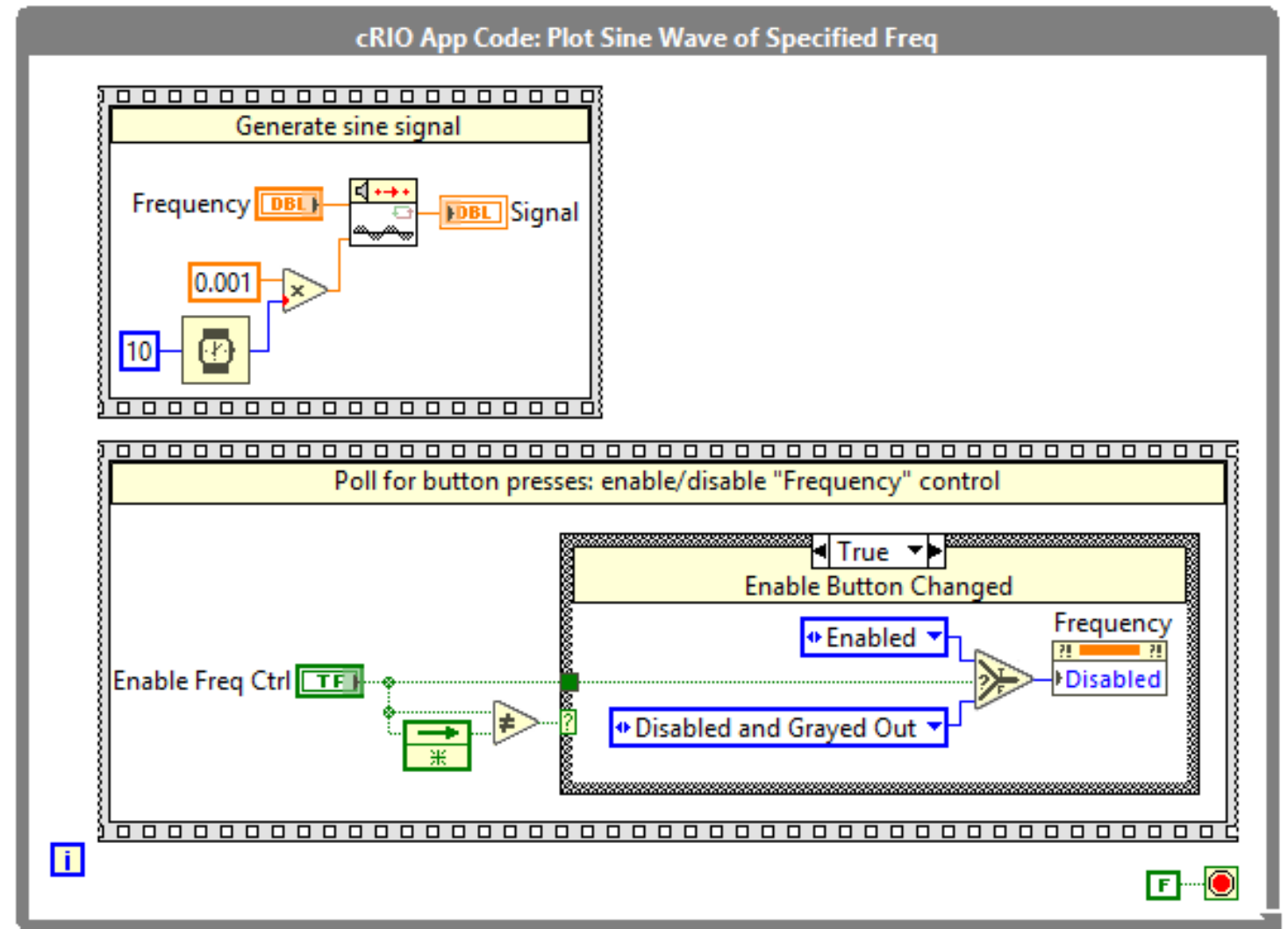
Option #2. Programmatically!



Demo

cRIO App Code

This technique also works for far larger and more complex apps

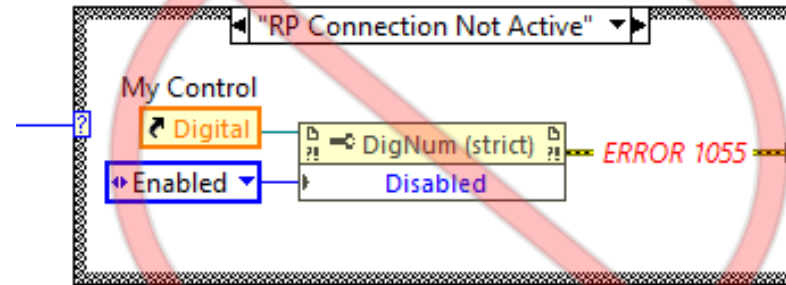


CAVEATS

Caveat #1: Be Careful Using UI Element Property Nodes

Can only call UI property node methods if there is currently an active Remote Panel connection.

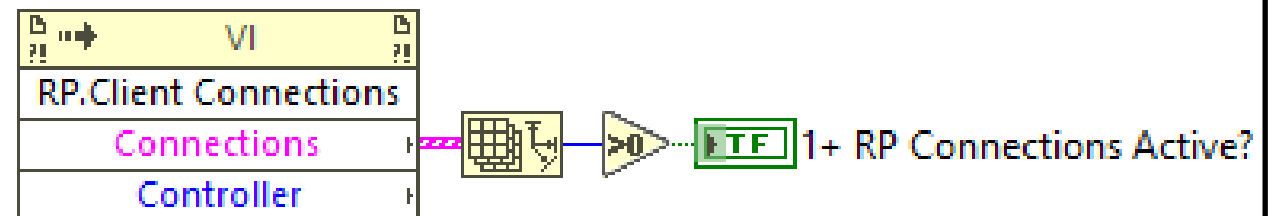
Otherwise, Error 1055 is thrown.



Solutions

1. Discard Error 1055 –**OR**–
2. Only call UI property nodes if connection active –**OR**–
3. Don't use UI property nodes

A VI running on a cRIO can detect if a Remote Panel connection is currently active to its own FP with the below code:



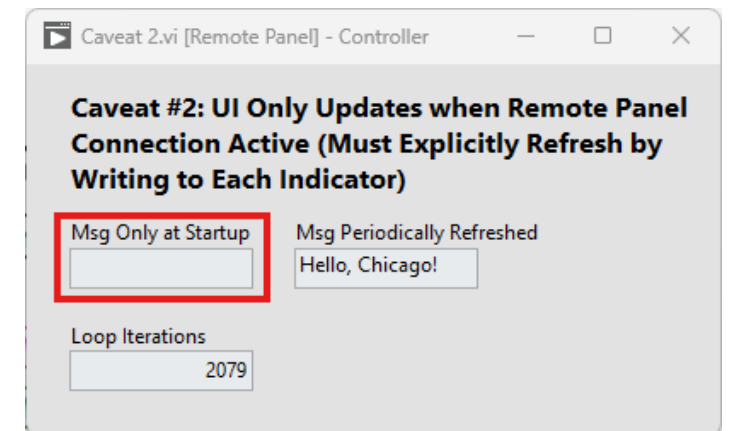
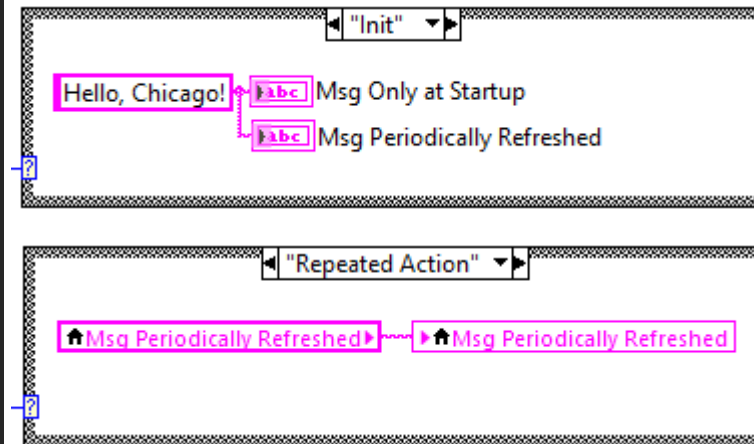
Caveat #2: Indicators May Need Refreshing

Front Panel indicators only refresh when they are written to at least once AFTER a remote panel connection has been made.

Otherwise, you just see the default value (the underlying variable in the running code will still be the correct value).

Solution

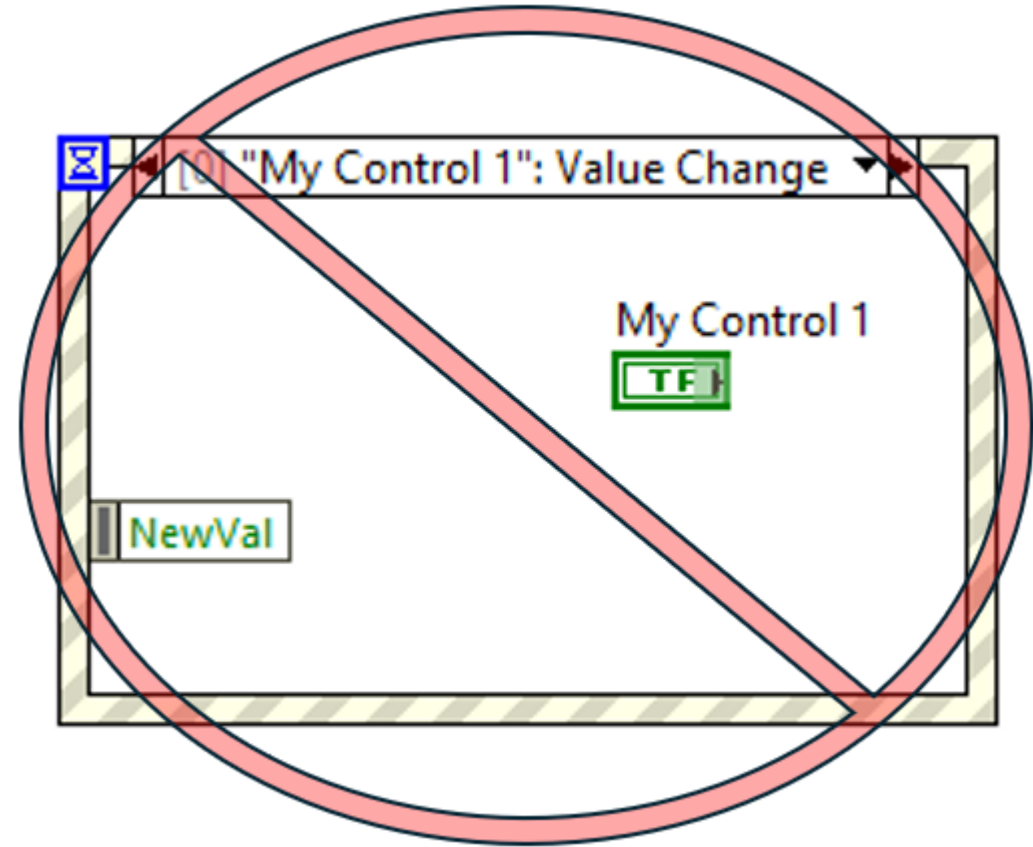
1. Write to UI indicators at least once after remote panel connection is established.



Caveat #3: Detect UI Events via Polling

Linux RT cRIOs* do not support static event (e.g. UI interaction) handling via event structure.

** Some newer models can handle UI events if enabled to do so in NI MAX settings.*



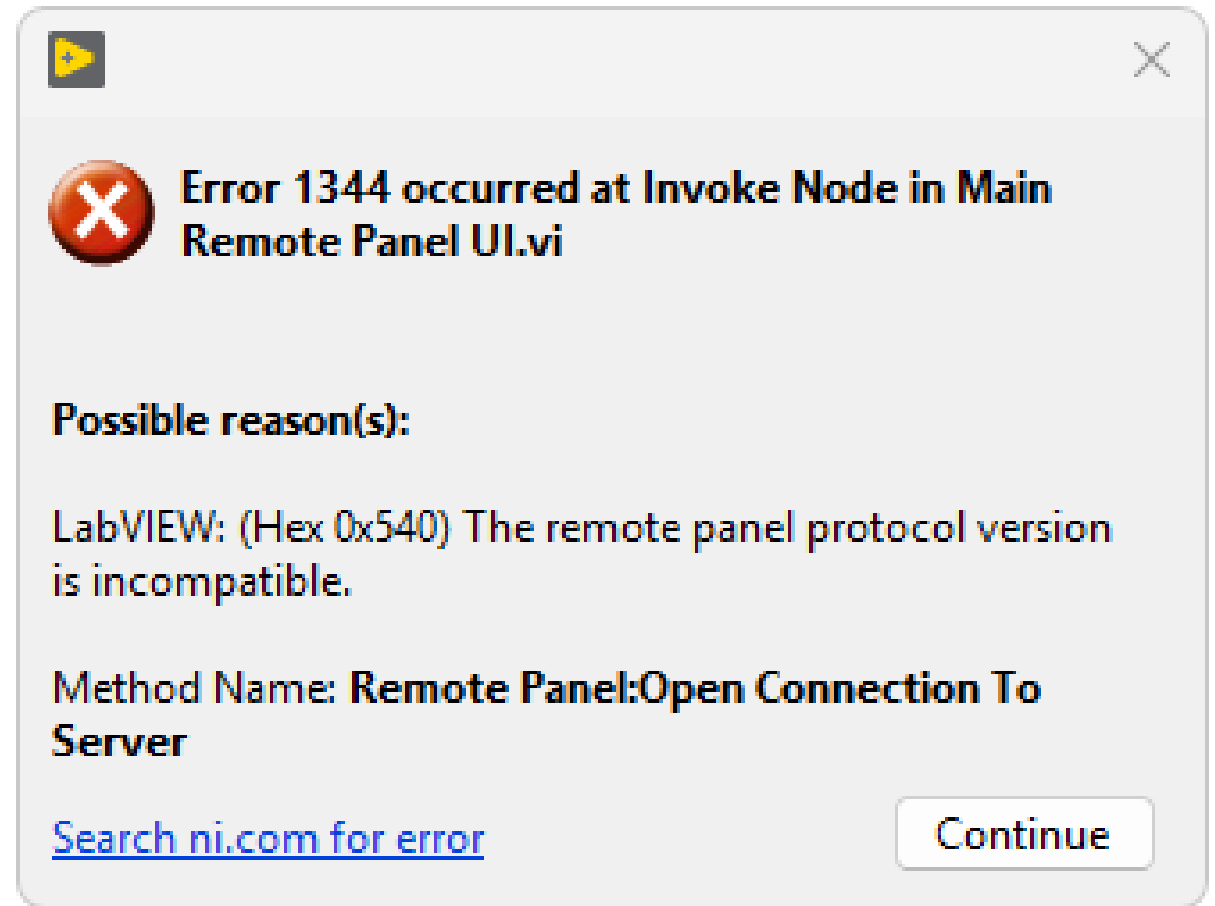
Solution

1. Use polling to detect UI control changes.

Caveat #4: Runtime Engine Mismatch

Client (PC) and Server (cRIO)
MUST have the same version of
the LabVIEW Runtime Engine.

Otherwise, Error 1344 is thrown.

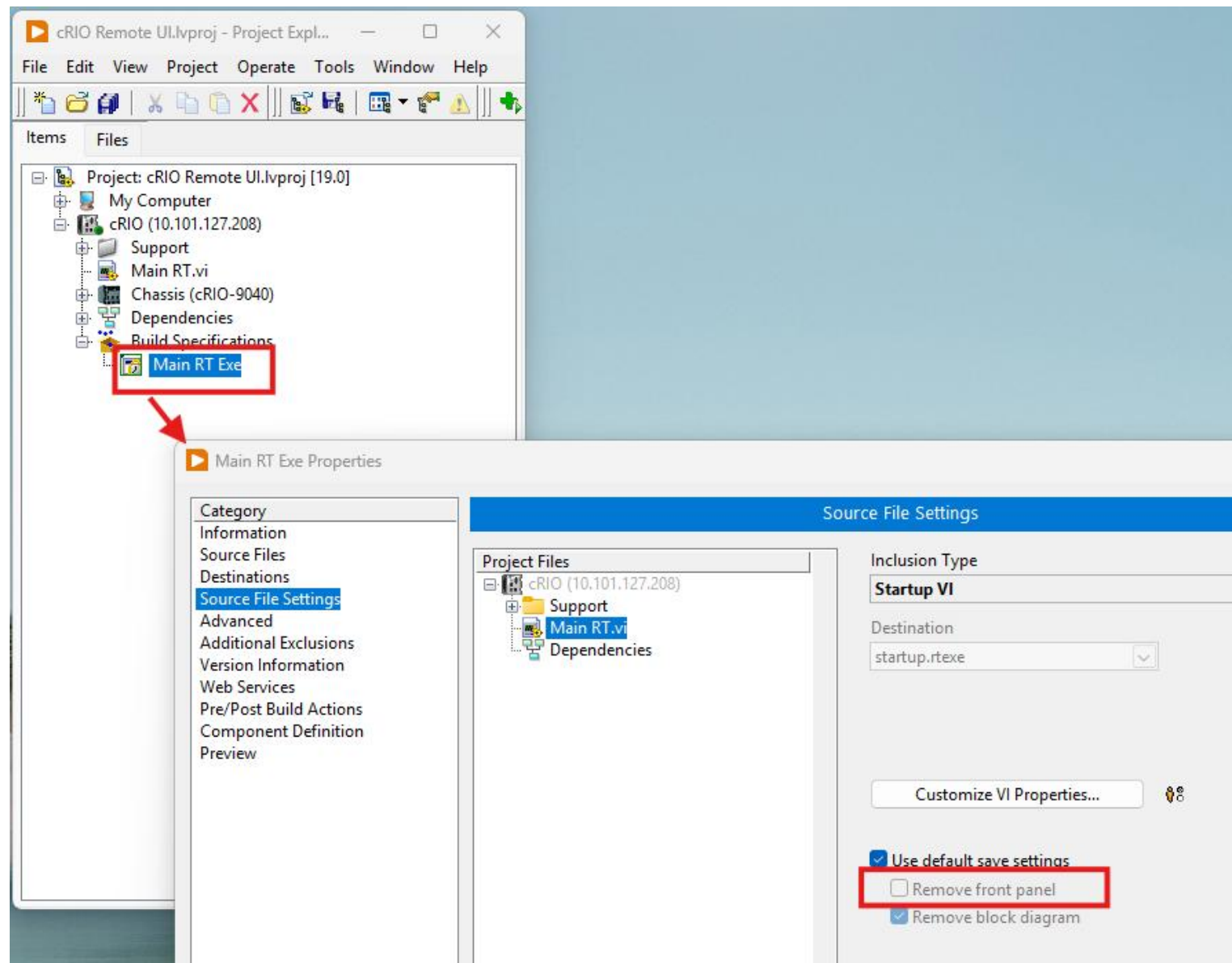


Solution

1. Make sure the LV RTE version matches on client and server.

Caveat #5: VI Front Panel Must Exist

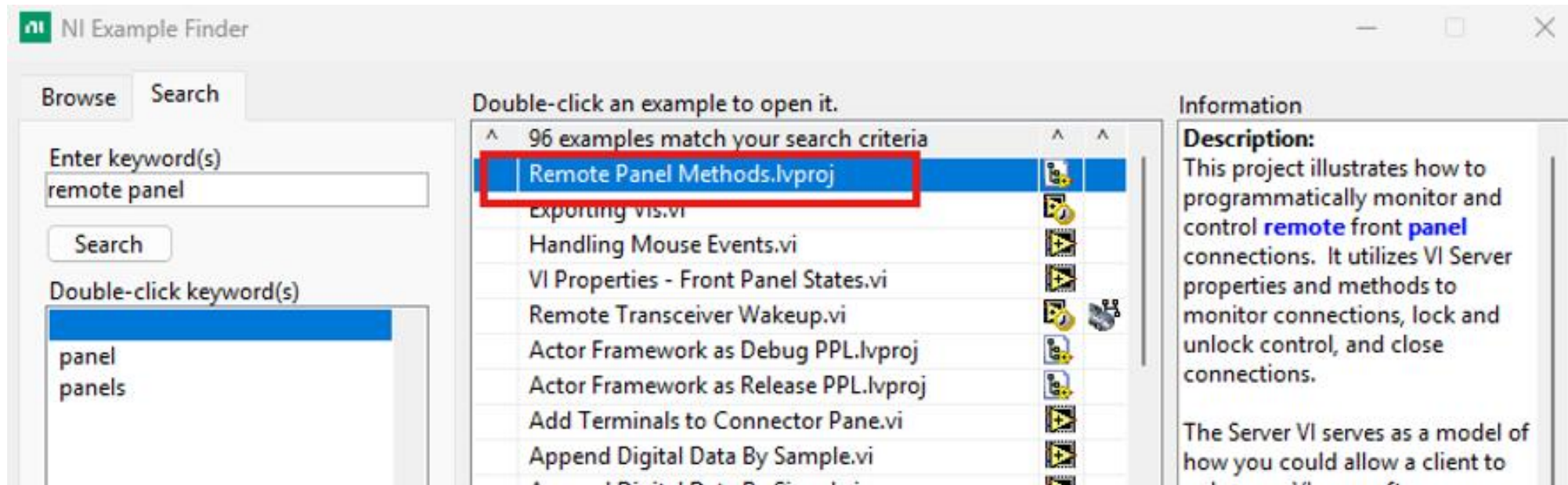
Make sure your cRIO app VI actually has a front panel!



Misc. Notes

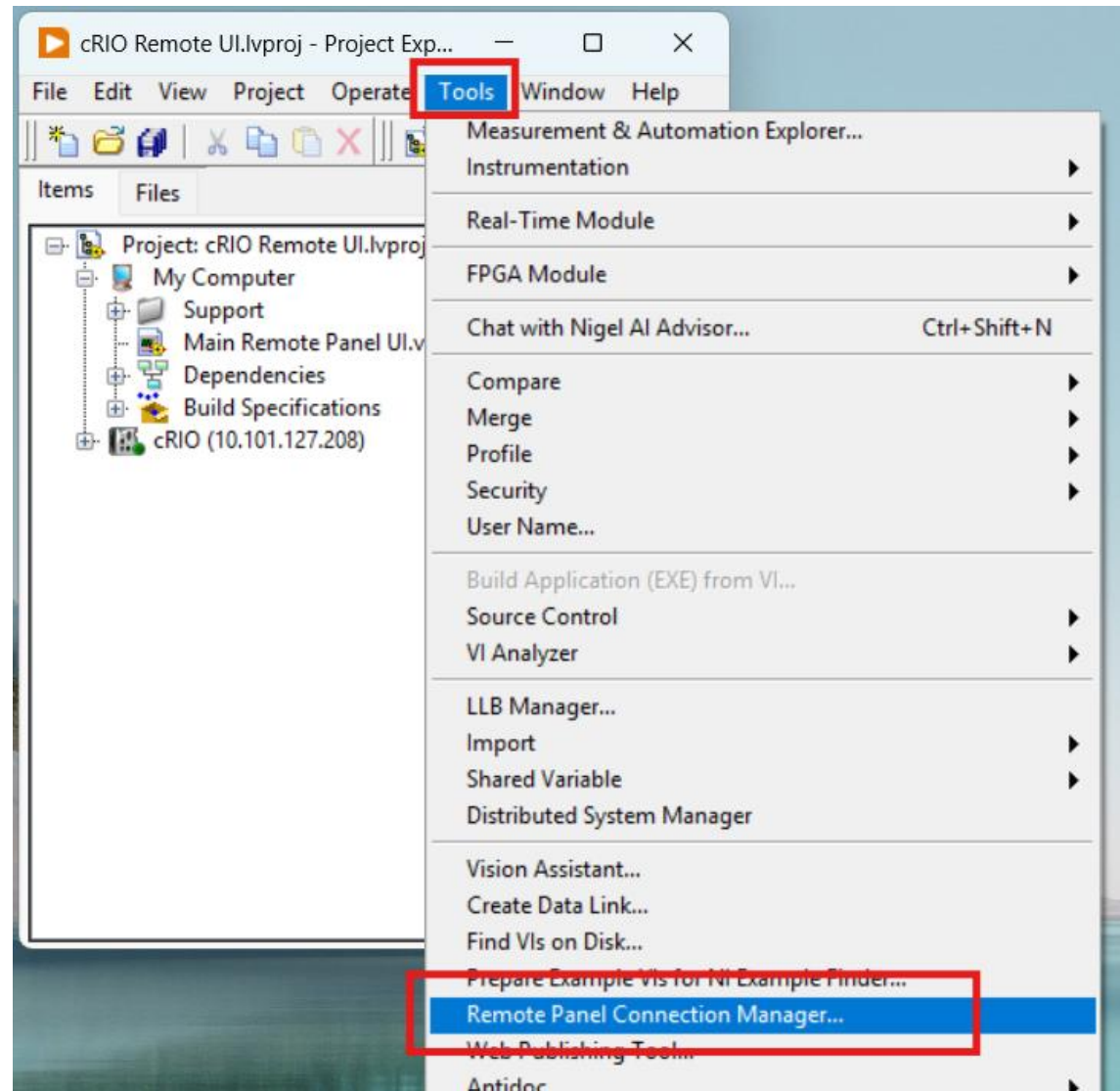
Misc. Notes

LabVIEW ships with an example remote panels project – just search for Remote Panels:



More Misc. Notes

Check out the Remote Panel
Connection Manager



Even More Misc. Notes

Debugging does not have to be enabled for this UI technique to work.

The cRIOs do not need to have Embedded UI capability.

I have used this technique as far back as LabVIEW 2021 and on 906X and 904x cRIOs running Linux RT.

See LabVIEW User Manual for more info: <https://www.ni.com/docs/en-US/bundle/labview/page/viewing-and-controlling-front-panels-remotely-with-the-web-server.html>

Example code is available here:

<https://github.com/JoeVonArx/LabVIEW-Remote-Panels>



[https://github.com/JoeVonArx/
LabVIEW-Remote-Panels](https://github.com/JoeVonArx/LabVIEW-Remote-Panels)

